

TRILHA BACKEND .NET

Plano de estudos + caderno de exercicios em C#

Objetivo	Stack principal	Metodo
Sair da logica e chegar a APIs .NET bem estruturadas.	C#, .NET, SQL, Git, Bash, Docker e arquitetura.	Aprender por exercicios, projetos e explicacao do por que.

Regra principal: nao avance porque "vi a materia". Avance quando conseguir resolver, depurar e explicar.

Nome: _____

Inicio: ____ / ____ / ____ Meta de sessoes por semana: _____

1. Como usar este caderno

Seu foco principal agora: C# e logica. Bash entra em paralelo para automacoes pequenas. Git entra desde o primeiro dia. Banco, Web API, Docker e arquitetura entram depois que a base estiver firme.

- Sessao de 60 min: 10 min revisao, 35 min exercicios, 10 min corrigir/refatorar, 5 min registrar o que aprendeu.
- Sessao de 30 min: 5 min revisao, 20 min um exercicio, 5 min explicar a solucao em voz alta ou em um README.
- Nao copie a solucao pronta. Primeiro tente, depois consulte documentacao, e so entao compare abordagens.
- Git desde o inicio: um repositorio chamado csharp-lab, uma pasta por nivel e commits pequenos.
- Regra de dominio: se voce acertou por sorte, ainda nao dominou. Refaça sem olhar no dia seguinte.

Estrutura sugerida do repositorio

```
csharp-lab/  
  01-fundamentos/  
  02-condicionais-loops/  
  03-metodos/  
  04-colecoes/  
  05-poo/  
  06-bcl-linq-arquivos/  
  07-async-http/  
  projetos/  
  bash/  
  README.md
```

Critério para passar de fase

- Resolver pelo menos 80% dos exercicios sem ajuda direta.
- Conseguir explicar entradas, saidas, casos-limite e complexidade basica da solucao.
- Conseguir encontrar um bug usando debugger, logs ou testes simples.
- Fazer pelo menos um mini-projeto da fase e versionar no Git.

2. Ordem de estudos corrigida

Fase	Assunto	O que dominar	Quando avançar
1	C# base	Tipos, operadores, fluxo, loops, metodos, arrays, strings	Resolve problemas sem travar na sintaxe
2	Git + Bash em paralelo	Commits, branches simples, scripts, pipes, arquivos, processos	Consegue automatizar tarefas e versionar tudo
3	POO + BCL	Classes, interfaces, excecoes, colecoes, IO, LINQ, JSON	Cria apps CLI organizados
4	SQL + banco	SELECT, JOIN, GROUP BY, INSERT/UPDATE/DELETE, modelagem	Consulta e modela dados sem ORM
5	Acesso a dados .NET	ADO.NET/EF Core, migrations, tracking, consultas	Liga app C# a um banco corretamente
6	.NET Web API	HTTP, REST, DTOs, DI, services, validacao, status codes	Cria API CRUD limpa e testavel
7	Testes	xUnit, unitarios, integracao, mocks com moderacao	Mudancas deixam de quebrar tudo
8	Docker	Images, containers, volumes, networks, compose	Sobe API + banco reproduzivelmente
9	Arquitetura	Camadas, dependencias, SOLID, modularidade, trade-offs	Organiza projetos maiores sem overengineering
10	Producao	CI/CD, seguranca, logs, observabilidade, cache, deploy	Projeto completo publicavel

Nota tecnica: Docker usa containers, nao maquinas virtuais tradicionais. Em Linux, containers normalmente compartilham o kernel do host; isso os deixa mais leves que uma VM.

O que faltava na sua lista

- Testes: voce vai precisar deles antes de arquitetura ficar realmente util.
- HTTP e redes: request/response, headers, status codes, DNS, portas e TLS.
- Seguranca: autenticacao, autorizacao, secrets, hash de senha, CORS e validacao.
- CI/CD: automatizar build, testes e deploy.
- Observabilidade: logs estruturados, metricas e tracing.
- Linux e deploy: processos, systemd, reverse proxy e ambiente de producao.

3. Ciclo sugerido de 12 semanas

Use as semanas como referencia, nao como prazo rigido. Se uma fase nao firmou, fique nela. O objetivo e competencia, nao velocidade.

Semanas	Foco	Entrega
1-2	C# fundamentos	Exercicios 01-24 + commits diarios
3	Metodos e decomposicao	Exercicios 25-34 + projeto calculadora CLI
4	Colecoes e strings	Exercicios 35-46 + analisador de texto
5	POO	Exercicios 47-61 + sistema de biblioteca
6	BCL, arquivos, JSON, LINQ	Exercicios 62-75 + task manager persistente
7	Async, HTTP + Bash	Exercicios 76-82 + scripts de automacao
8	SQL	Consultas + modelagem + SQLite/PostgreSQL
9	EF Core	Persistencia do projeto em banco
10	.NET Web API	API CRUD com DTOs e DI
11	Testes + Docker	xUnit + Docker Compose com banco
12	Projeto final	API completa, README, GitHub e arquitetura explicada

4. Exercícios C# - Nível 1: sintaxe, tipos e operadores

Objetivo: ficar confortável com Console, variáveis, tipos, conversões e expressões. Faça cada exercício em um arquivo/projeto separado ou por pastas.

01. Ola, usuario [fundamentos]

Leia o nome do usuário e imprima uma saudação personalizada.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

02. Idade futura [fundamentos]

Leia a idade atual e mostre qual será a idade daqui a 5, 10 e 20 anos.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

03. Conversor de temperatura [fundamentos]

Converta Celsius para Fahrenheit e Kelvin. Valide a entrada numérica.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

04. Média de notas [fundamentos]

Leia 4 notas, calcule média simples e imprima com duas casas decimais.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

05. Troca de valores [fundamentos]

Leia dois inteiros e troque seus valores. Faça primeiro com variável auxiliar e depois usando tupla.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

06. IMC matemático [fundamentos]

Leia peso e altura e calcule o IMC. Neste exercício, foque no cálculo e formatação, não em diagnóstico de saúde.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

07. Segundos para horário [fundamentos]

Converta um total de segundos em horas, minutos e segundos.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

08. Dinheiro [fundamentos]

Leia preço e quantidade, calcule subtotal, desconto percentual e total final.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

09. Divisão segura [fundamentos]

Leia dois números e evite divisão por zero. Mostre quociente e resto quando fizer sentido.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

10. Bit a bit [fundamentos]

Receba dois inteiros pequenos e mostre AND, OR, XOR e deslocamentos. Escreva no README o que cada operador faz.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

11. Par ou ímpar [fundamentos]

Dado um inteiro, determine se é par ou ímpar usando operador módulo.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

12. Casas de um número [fundamentos]

Receba um inteiro positivo de 0 a 9999 e separe unidade, dezena, centena e milhar usando divisão e módulo.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

Checkpoint 1

- Você sabe a diferença entre int, long, float, double e decimal?

- Sabe quando usar TryParse em vez de confiar em Parse?
- Consegue prever o resultado de uma expressao antes de rodar o codigo?

5. Exercicios C# - Nivel 2: condicionais e loops

13. Classificador de numero [if/switch | loops]

Leia um numero e classifique como positivo, negativo ou zero; depois diga se e par ou impar.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

14. Maior de tres [if/switch | loops]

Leia tres numeros e encontre maior e menor sem usar LINQ.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

15. Calculadora com menu [if/switch | loops]

Use switch para escolher soma, subtracao, multiplicacao, divisao ou potencia.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

16. Ano bissexto [if/switch | loops]

Receba um ano e diga se ele e bissexto. Pesquise e escreva a regra correta antes de codar.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

17. Preco por faixa [if/switch | loops]

Aplique descontos diferentes conforme o valor de uma compra. Evite condicoes sobrepostas.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

18. Tabuada [if/switch | loops]

Mostre a tabuada de 1 a 10 para um numero informado.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

19. Somatorio [if/switch | loops]

Some todos os inteiros de 1 ate N. Faça com loop e depois compare com a formula matematica.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

20. Fatorial [if/switch | loops]

Calcule N! para valores pequenos. Descubra em que ponto int e long estouram.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

21. Contagem regressiva [if/switch | loops]

Mostre uma contagem regressiva de N ate 0 e imprima "GO!" no final.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

22. Numeros primos [if/switch | loops]

Determine se um numero e primo. Depois liste os primos ate N.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

23. MDC e MMC [if/switch | loops]

Implemente o algoritmo de Euclides para MDC e derive o MMC.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

24. Fibonacci [if/switch | loops]

Gere os N primeiros termos. Depois faca uma versao que pare ao ultrapassar um limite.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

25. Adivinhacao [if/switch | loops]

Gere um numero aleatorio e permita tentativas ate acertar. Informe "maior" ou "menor".

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

26. Menu persistente [if/switch | loops]

Crie um menu que so encerra quando o usuario escolher sair. Cada opcao chama uma operacao diferente.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

27. Padroes com loops [if/switch | loops]

Desenhe triangulos de asteriscos, crescente e decrescente, usando loops aninhados.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

28. Caixa eletrônico [if/switch | loops]

Dado um valor inteiro, informe quantas notas de 100, 50, 20, 10, 5 e 2 seriam usadas numa estratégia gulosa.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

6. Exercicios C# - Nivel 3: metodos e decomposicao

29. Metodo IsPrime [metodos]

Transforme o teste de primo em `bool IsPrime(int n)` e escreva casos de teste manuais.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

30. Metodos de calculadora [metodos]

Cada operacao da calculadora deve virar um metodo. O Main so coordena entrada, chamada e saida.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

31. Conversor generico de notas [metodos]

Crie metodos para media, maior nota, menor nota e situacao final.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

32. Validador de entrada [metodos]

Crie um metodo que continue perguntando ate receber um inteiro valido dentro de um intervalo.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

33. Sobrecarga [metodos]

Crie metodos Area sobrecarregados para quadrado, retangulo e circulo. Depois explique a limitacao dessa ideia.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

34. Parametros opcionais [metodos]

Crie um metodo de formatacao de nome com parametros opcionais para prefixo e caixa alta/baixa.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

35. Tuplas [metodos]

Escreva um metodo que receba uma sequencia de numeros e retorne minimo, maximo e media em uma tupla.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

36. Recursao controlada [metodos]

Implemente fatorial recursivo e iterativo. Compare legibilidade e risco de stack overflow.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

37. Busca binaria [metodos]

Implemente busca binaria em array ordenado sem usar metodos prontos.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

38. Refatoracao [metodos]

Pegue um exercicio anterior com Main grande e divida em metodos pequenos com nomes que expressem intencao.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

Mini-projeto A - Calculadora CLI 2.0

- Menu em loop, validacao de entrada e metodos separados.
- Historico das ultimas operacoes em memoria.
- Tratamento de divisao por zero e entradas invalidas.
- README explicando estrutura, decisoes e exemplos de uso.
- Commits sugeridos: setup, operacoes basicas, validacao, historico, refactor.

7. Exercicios C# - Nivel 4: arrays, listas, dicionarios e strings

39. Maior e menor em array [colecoes | strings]

Leia N numeros e encontre maior, menor, soma e media sem LINQ.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

40. Inverter array [colecoes | strings]

Inverta um array manualmente trocando elementos das pontas para o centro.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

41. Remover duplicados [colecoes | strings]

Receba uma lista de inteiros e produza outra sem repetidos. Faça sem LINQ e depois com HashSet.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

42. Frequencia de numeros [colecoes | strings]

Conte quantas vezes cada inteiro aparece usando Dictionary.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

43. Segundo maior [colecoes | strings]

Encontre o segundo maior valor distinto de uma lista. Defina o comportamento quando ele nao existir.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

44. Rotacao [colecoes | strings]

Rotacione uma lista K posicoes para a direita. Pense em K maior que o tamanho.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

45. Palindromo [colecoes | strings]

Verifique se uma frase e palindromo ignorando espacos, pontuacao e diferenca entre maiusculas/minusculas.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

46. Contador de palavras [colecoes | strings]

Conte palavras e frequencias em um texto. Mostre as 5 mais comuns.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

47. Anagrama [colecoes | strings]

Verifique se duas palavras/frases sao anagramas depois de normalizar o texto.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

48. Cifra de Cesar [colecoes | strings]

Implemente cifrar e decifrar deslocando letras. Preserve caracteres nao alfabeticos.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

49. Agenda simples [colecoes | strings]

Use Dictionary para nome -> telefone, com menu adicionar, buscar, listar e remover.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

50. Fila de atendimento [colecoes | strings]

Modele uma fila com Queue. Adicionar pessoa, chamar proxima e mostrar tamanho.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

51. Historico undo [colecoes | strings]

Use Stack para manter acoes e permitir desfazer a ultima.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

52. Matriz [colecoes | strings]

Some duas matrizes pequenas e depois calcule a transposta de uma matriz.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

53. Jogo da velha - estado [colecoes | strings]

Represente o tabuleiro em matriz/array e escreva funcoes para validar jogada e detectar vencedor.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

8. Exercicios C# - Nivel 5: POO

Aqui a meta nao e "usar classe porque sim". A meta e modelar estado, comportamento e responsabilidades de forma que o codigo fique mais facil de mudar.

54. Classe ContaBancaria [POO]

Crie saldo privado, titular e metodos Depositar/Sacar. Nao permita saldo invalido por acesso direto.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

55. Construtores [POO]

Crie uma classe Produto com construtor obrigando nome e preco validos. Adicione sobrecarga quando fizer sentido.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

56. Encapsulamento [POO]

Crie uma propriedade de idade que rejeite valores negativos e outra calculada somente leitura.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

57. Composicao [POO]

Uma Turma possui Alunos. Calcule media da turma sem expor internamente uma lista mutavel desnecessariamente.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

58. Heranca basica [POO]

Modele Funcionario, Desenvolvedor e Gerente. Depois escreva por que composicao pode ser melhor em alguns cenarios.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

59. Polimorfismo [POO]

Crie formas com metodo Area e trate varias formas por uma abstracao comum.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

60. Interface [POO]

Crie INotificador com implementacoes Email e Console. Um servico deve depender da interface.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

61. Classe abstrata [POO]

Modele meios de pagamento com regras comuns e comportamento especifico. Compare com interface.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

62. Equals e identidade [POO]

Crie Value Object simples para Email ou CodigoProduto e defina igualdade por valor.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

63. Records [POO]

Modele um DTO imutavel com record e compare com class mutavel.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

64. Generics [POO]

Implemente uma classe RepositorioEmMemoria<T> com adicionar, listar e remover.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

65. Strategy [POO]

Crie estrategias diferentes de desconto e escolha em runtime sem grande switch central.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

66. Dependency Injection manual [POO]

Passe dependencias pelo construtor em vez de criar tudo com new dentro das classes de regra.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

67. Sistema de biblioteca [POO]

Classes Livro, Usuario e Emprestimo. Defina regras: disponibilidade, limite de emprestimos e devolucao.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

68. Refatoracao SOLID [POO]

Pegue uma classe que faz entrada, regra, salvamento e impressao. Separe responsabilidades e explique cada dependencia.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

Mini-projeto B - Sistema de biblioteca CLI

- Cadastro de livros e usuarios.
- Emprestimo/devolucao com regras de negocio.
- Persistencia inicialmente em memoria.
- Interfaces para repositório e saída; implementações concretas separadas.
- Testar manualmente casos normais e casos-limite.
- Bonus: depois salvar em JSON sem alterar a regra de negocio.

9. Exercicios C# - Nivel 6: BCL, arquivos, JSON e LINQ

As "bibliotecas basicas do C#" sao, na pratica, principalmente APIs da Base Class Library do .NET: Collections, IO, LINQ, JSON, HTTP, Tasks e outras.

69. Leitor de arquivo [BCL | .NET]

Leia um arquivo texto e mostre numero de linhas, palavras e caracteres.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

70. Copiador com seguranca [BCL | .NET]

Copie um arquivo para outra pasta e trate arquivo inexistente, permissao e destino ja existente.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

71. Scanner de extensoes [BCL | .NET]

Percorra uma pasta e conte arquivos por extensao usando System.IO.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

72. Log parser [BCL | .NET]

Leia um arquivo de log e conte linhas INFO, WARN e ERROR. Liste as mensagens ERROR.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

73. Serializacao JSON [BCL | .NET]

Salve uma lista de tarefas em JSON e recupere na proxima execucao.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

74. Configuracao [BCL | .NET]

Leia configuracoes de um arquivo JSON e aplique valores padrao quando campos nao existirem.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

75. LINQ - filtros [BCL | .NET]

Dada uma lista de produtos, filtre por faixa de preco, ordene por nome e projete apenas nome/preco.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

76. LINQ - agrupamento [BCL | .NET]

Agrupe vendas por categoria e calcule total e media por grupo.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

77. LINQ - joins em memoria [BCL | .NET]

Junte clientes e pedidos por id usando Join. Compare com loops aninhados.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

78. LINQ - armadilha [BCL | .NET]

Crie uma consulta LINQ e altere a colecao antes de enumerar. Explique deferred execution.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

79. DateTime [BCL | .NET]

Calcule diferenca entre datas, inicio/fim do mes e proximos 5 dias uteis simples.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

80. Regex basica [BCL | .NET]

Extraia emails e numeros de telefone de um texto de teste. Registre limitacoes da regex.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

81. Exceptions [BCL | .NET]

Crie uma excecao de dominio para uma regra do sistema de biblioteca e trate somente onde fizer sentido.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

82. using/IDisposable [BCL | .NET]

Abra um arquivo/stream e garanta liberacao de recurso. Explique por que using existe.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

83. HashSet [BCL | .NET]

Compare duas listas de tags e encontre intersecao, uniao e diferenca.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

10. Exercicios C# - Nivel 7: async, HTTP e concorrencia basica

84. Delay sem bloquear [async | HTTP]

Crie um contador que usa await Task.Delay em vez de Thread.Sleep e explique a diferenca conceitual.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

85. Download HTTP [async | HTTP]

Use HttpClient para fazer GET em uma API publica de teste e imprimir status code e parte da resposta.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

86. JSON HTTP [async | HTTP]

Desserialize uma resposta JSON para records/classes tipadas.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

87. Chamadas em paralelo [async | HTTP]

Dispare 3 requisicoes independentes e aguarde com Task.WhenAll. Compare o tempo com chamadas sequenciais.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

88. Cancelamento [async | HTTP]

Use CancellationToken para cancelar uma operacao longa de forma cooperativa.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

89. Retry consciente [async | HTTP]

Simule operacao instavel e implemente poucas tentativas com espera. Explique por que retry infinito e perigoso.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

90. Producer/consumer simples [async | HTTP]

Modele produtor e consumidor com Channel ou estrutura equivalente, apenas como desafio avancado.

☐ Feito ☐ Revisei ☐ Consigo explicar sem olhar

Mini-projeto C - Task Manager CLI persistente

- Tarefas com id, titulo, descricao, prioridade, status e data.
- Comandos: adicionar, listar, concluir, remover, filtrar e buscar.
- Persistencia em JSON.
- LINQ para filtros e ordenacao.
- Arquitetura simples: dominio, servico, repositorio, UI de console.
- Depois, este projeto pode evoluir para banco e Web API.

11. Bash em paralelo - 12 desafios de automacao

Bash aqui e ferramenta de automacao do ambiente. Nao precisa virar seu foco principal. Duas sessoes curtas por semana ja bastam no inicio.

B01. Hello arguments

Script que recebe nome por argumento e mostra ajuda se estiver faltando.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B02. Backup

Compacte uma pasta com data/hora no nome e salve em backups/.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B03. Limpeza

Remova arquivos temporarios mais antigos que N dias de uma pasta de teste.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B04. Project bootstrap

Crie pastas src, tests, docs e um README automaticamente.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B05. Dotnet runner

Script que executa dotnet format, dotnet build e dotnet test; pare ao primeiro erro.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B06. Log filter

Leia um log e mostre apenas ERROR usando grep; depois conte ocorrencias.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B07. Uso de disco

Mostre particoes e alerte se alguma ultrapassar um limite configuravel.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B08. Process checker

Verifique se um processo esta rodando e retorne exit code adequado.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B09. Env validator

Cheque se dotnet, git e docker existem no PATH e imprima versoes.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B10. Batch rename

Renomeie arquivos de teste seguindo um padrao; tenha modo dry-run antes de alterar.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B11. Git helper

Mostre branch atual, status curto e ultimos 5 commits.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

B12. Build artifact

Publique uma app .NET para uma pasta dist/ e gere um arquivo compactado.

☐ Feito ☐ Entendi pipes/exit codes envolvidos

12. Git e GitHub - use desde o primeiro dia

- Inicializar repositório, adicionar .gitignore e escrever README.
- Commits pequenos com mensagem que diga o que mudou, não "update".
- Criar branch para um mini-projeto/refatoração e fazer merge.
- Aprender status, diff, log, restore, switch, branch, merge e rebase mais tarde.
- Subir projetos no GitHub com README, instruções de execução e screenshots/requests quando houver API.

Desafios Git

- Faça uma mudança errada e use diff para identificá-la antes do commit.
- Recupere um arquivo modificado sem querer.
- Crie um conflito de merge proposital em um repositório de teste e resolva.
- Use tags para marcar v0.1, v0.2 e v1.0 de um mini-projeto.
- Escreva um README que outra pessoa consiga seguir sem falar com você.

13. Depois do C#: SQL, .NET Web API, Docker e arquitetura

Banco de dados

- Comece por SQL puro: tabelas, chaves, constraints, SELECT, WHERE, ORDER BY, GROUP BY, JOIN, subqueries e transacoes.
- Use SQLite para praticar localmente e PostgreSQL quando quiser um banco servidor mais proximo de producao.
- Somente depois entre forte em EF Core. ORM nao substitui entender SQL.

Exercicios de banco para o seu Task Manager

- Modele tabelas de usuarios, tarefas, tags e relacao tarefa-tag.
- Escreva consultas para tarefas atrasadas, concluidas por periodo e quantidade por prioridade.
- Crie indices e explique que consulta eles pretendem acelerar.
- Implemente migration no EF Core e depois inspecione o SQL gerado.

.NET Web API

- Aprenda HTTP antes de decorar atributos: metodos, status codes, headers, JSON, idempotencia e REST.
- Depois: Minimal APIs ou Controllers, DTOs, validacao, dependency injection, services, repositories, middleware, logging e OpenAPI.
- Autenticacao/autorizacao entra depois do CRUD estar solido.

Projeto de API

- GET /tasks, GET /tasks/{id}, POST /tasks, PUT/PATCH e DELETE.
- Filtros por status/prioridade e paginacao.
- DTOs separados das entidades de persistencia.
- Validacao e respostas com status code correto.
- Testes de integracao para endpoints principais.

Testes

- xUnit para regras de dominio e servicos.
- Teste de integracao para banco/API.
- Aprenda Arrange-Act-Assert, test cases e nomes descritivos.
- Nao use mock em tudo: teste comportamento e mantenha os testes legiveis.

Docker

- Dockerfile multi-stage para a API.
- Docker Compose com API + PostgreSQL.
- Volumes para persistencia e networks para comunicacao entre containers.
- Variaveis de ambiente e secrets fora da imagem.
- Healthcheck e logs como parte do ambiente.

Arquitetura de software

Arquitetura não é decorar Clean Architecture. É decidir limites, dependências e trade-offs para o projeto continuar modificável.

- Comece com separação simples: domínio, aplicação/serviços, infraestrutura e interface/API.
- Aprenda acoplamento, coesão, dependency inversion e SOLID com exemplos concretos.
- Depois estude modular monolith, Clean Architecture, DDD básico, CQRS e mensageria somente quando houver problema real que justifique.

14. Projetos que vao costurar a trilha

Projeto	Tecnologias	Objetivo
1. Calculadora CLI 2.0	C# + Git	Sintaxe, metodos, validacao e commits
2. Biblioteca CLI	C# POO + JSON	Modelagem, interfaces, persistencia simples
3. Task Manager CLI	C# + LINQ + IO	Aplicacao organizada e persistente
4. Task Manager DB	SQL + EF Core	Persistencia real e migrations
5. Task Manager API	.NET Web API + xUnit	HTTP, DTOs, DI, testes
6. Ambiente reproduvel	Docker + Bash	API + banco subindo com um comando
7. Projeto final	.NET + PostgreSQL + Docker + CI	Backend completo pronto para portfolio

Projeto final sugerido: API de gestao de estudos

- Usuarios, materias, topicos, sessoes de estudo, metas e progresso.
- PostgreSQL + EF Core.
- Autenticacao depois do CRUD principal.
- Filtros, paginacao, validacao e logs.
- Testes unitarios e de integracao.
- Docker Compose para desenvolvimento.
- GitHub Actions para build/test.
- README explicando arquitetura, como subir e decisoes tecnicas.

15. Checklist de dominio

C# base

- [] Tipos e conversoes Nota (0-5): ____ Evidencia/projeto: _____
- [] Condicionais e switch Nota (0-5): ____ Evidencia/projeto: _____
- [] Loops Nota (0-5): ____ Evidencia/projeto: _____
- [] Metodos Nota (0-5): ____ Evidencia/projeto: _____
- [] Arrays/listas/dicionarios Nota (0-5): ____ Evidencia/projeto: _____
- [] Strings Nota (0-5): ____ Evidencia/projeto: _____
- [] Debugging Nota (0-5): ____ Evidencia/projeto: _____

C# intermediario

- [] POO Nota (0-5): ____ Evidencia/projeto: _____
- [] Interfaces Nota (0-5): ____ Evidencia/projeto: _____
- [] Generics Nota (0-5): ____ Evidencia/projeto: _____
- [] Exceptions Nota (0-5): ____ Evidencia/projeto: _____
- [] LINQ Nota (0-5): ____ Evidencia/projeto: _____
- [] IO/JSON Nota (0-5): ____ Evidencia/projeto: _____
- [] async/await Nota (0-5): ____ Evidencia/projeto: _____
- [] HttpClient Nota (0-5): ____ Evidencia/projeto: _____

Backend

- [] SQL Nota (0-5): ____ Evidencia/projeto: _____
- [] EF Core Nota (0-5): ____ Evidencia/projeto: _____
- [] HTTP/REST Nota (0-5): ____ Evidencia/projeto: _____
- [] Web API Nota (0-5): ____ Evidencia/projeto: _____
- [] DI Nota (0-5): ____ Evidencia/projeto: _____
- [] Validacao Nota (0-5): ____ Evidencia/projeto: _____
- [] Testes Nota (0-5): ____ Evidencia/projeto: _____

Entrega

- [] Git/GitHub Nota (0-5): ____ Evidencia/projeto: _____
- [] Bash Nota (0-5): ____ Evidencia/projeto: _____
- [] Docker Nota (0-5): ____ Evidencia/projeto: _____
- [] CI/CD Nota (0-5): ____ Evidencia/projeto: _____
- [] Logs Nota (0-5): ____ Evidencia/projeto: _____
- [] Seguranca basica Nota (0-5): ____ Evidencia/projeto: _____
- [] Arquitetura Nota (0-5): ____ Evidencia/projeto: _____

Escala de 0 a 5

- 0: nunca vi.

- 1: reconheço os termos.
- 2: consigo fazer com muita consulta.
- 3: resolvo problemas comuns sozinho.
- 4: depuro, explico e escolho entre alternativas.
- 5: consigo ensinar, revisar código e lidar com casos menos óbvios.

16. Registro de sessoes

No fim de cada sessao, escreva uma frase sobre o que voce entendeu e uma sobre o que ainda esta confuso. Isso transforma "estudei" em evidencia real de progresso.

Data	Tema	Exercicios/Projeto	O que aprendi / duvida

17. Regras para não se perder

- Não pule de linguagem em linguagem. Sua stack principal é C#/.NET; Python/Bash ficam como ferramentas auxiliares.
- Não comece por arquitetura pesada. Primeiro escreva programas, depois sinta os problemas de organização que a arquitetura resolve.
- Não deixe Git para depois. Versionamento faz parte do trabalho, não é matéria separada.
- Não use EF Core para esconder SQL. Aprenda consultas e modelagem primeiro.
- Não use Docker como magia. Entenda processo, porta, arquivo, volume, rede e variável de ambiente.
- Projeto > tutorial infinito. A cada bloco de teoria, produza algo executável.
- Explique o por que. Se você sabe dizer por que escolheu uma lista, interface, transação ou status code, está saindo da decoração para engenharia.

Meta final desta trilha: conseguir criar, testar, versionar, containerizar e explicar uma API .NET com banco de dados sem depender de um passo a passo.